

1. Executive Summary & System Overview

CLAB (C Learning & Assessment Buddy) is a dedicated full-stack web application designed to help engineering and computer science students build foundational logic and programming proficiency in the C language. The platform combines daily problem challenges (Problem of the Day / POTD), automated evaluation against hidden test cases, visual streak tracking, and an administrative control portal.

Student Learning Portal

Interactive browser-based code editing, daily problem solving, progress & streak tracking,

and complete submission history with execution status.

Admin Management Portal

Problem lifecycle management (CRUD, drafting, publishing, constraints, test cases),

student directory inspection, and activity analytics.

2. High-Level Architectural Diagram

+-----+ |

FRONTEND CLIENT (React 19 + Vite) | | | [Public Portal] [Student Portal] [Admin

Portal] | | - Landing Page - /dashboard - /admin (Dashboard) | | - /login, /register -

/today (POTD) - /admin/problems | | - /problems & /problems/:id - /admin/problems/new | | -

/submissions & /statistics - /admin/students | | | AppRoutes (Role Guards:

ProtectedRoute, AdminRoute, Smart Fallback) | | Global Auth Context (JWT Storage, User

State) & Axios HTTP Interceptors | +-----

+-----+ | JSON over REST (Bearer JWT Auth) v

+-----+ |

BACKEND SERVER (Node.js + Express.js) | | | CORS Middleware | Express JSON Parser | Auth

Middleware | Admin Role Guard | | | [Controllers] | | - userController : Registration,

Authentication & JWT Generation | | - problemController : POTD, Problem CRUD, Filtering &

Publishing | | - adminController : Admin Stats Aggregation & Student Directory | | -

submissionController : C Code Submissions & Result Tracking | | - progressController :

Streaks & Problem-Solving Metrics | | - topicController : Module Taxonomy & Categorization

| +-----+-----+ |

Mongoose ODM v

+-----+ |

DATABASE (MongoDB Atlas) | | | Users Collection | Problems Collection | Submissions

Collection | | Progress Collection | Topics Collection |

+-----+

3. Technology Stack Specification

Layer	Technologies & Libraries	Role in Architecture
Frontend Core	React 19, Vite 8, ES Modules	High-performance single page application (SPA) rendering and fast client-side HMR.
Styling & Icons	Tailwind CSS v4, Lucide React	Modern responsive layouts, dark/light contrast cards, standardized icon sets.
Client Routing	React Router DOM v7	Nested portal layouts (StudentLayout , AdminLayout), route guards, role redirection.
State & Auth	React Context API, LocalStorage	Global session persistence, user role detection, token handling.
Backend Runtime	Node.js v22, Express.js 4	REST API service, request validation, business logic, modular routing.
Database &	MongoDB Atlas, Mongoose 8	Document storage, schema validation,

Layer	Technologies & Libraries	Role in Architecture
ODM		relationship referencing (<code>populate</code>).
Security & Crypto	JSON Web Tokens (<code>jsonwebtoken</code>), <code>bcryptjs</code> , CORS	Signed bearer tokens (7-day lifespan), salt-hashed passwords (10 rounds).

4. Database Schema & Data Models

The MongoDB database utilizes five core collections structured with Mongoose schemas:

4.1 User Schema (`models/User.js`)

Field	Type	Constraints	Description
<code>username</code>	String	Required, Trim, 3-30 chars	Display name of student or administrator
<code>rollNo</code>	String	Required, Unique, Trim	Academic identification number
<code>email</code>	String	Required, Unique, Lowercase	Login and communication email
<code>password</code>	String	Required, Hashed	Bcrypt hashed credential string
<code>role</code>	String	Enum: ['student', 'admin']	Role-based access classification (default: 'student')
<code>currentStreak</code>	Number	Default: 0	Active consecutive days with accepted submissions
<code>longestStreak</code>	Number	Default: 0	Historical peak streak achieved by user

4.2 Problem Schema (`models/Problem.js`)

Field	Type	Constraints	Description
<code>title</code>	String	Required, Trim	Problem headline
<code>description</code>	String	Required	Full problem statement and instructions

Field	Type	Constraints	Description
topic	String	Enum: Basics, Loops, Arrays...	Subject categorization
difficulty	String	Enum: ['Easy', 'Medium', 'Hard']	Complexity tier
testCases	Array	[{ input, expectedOutput, isHidden }]	Test cases for automated grading
status	String	Enum: ['draft', 'published']	Visibility control (default: 'draft')
publishDate	Date	Required	Date designated for POTD delivery
createdBy	ObjectId	Ref: User	Admin who authored the problem

5. REST API Architecture & Endpoints

Method	Endpoint	Access Level	Description
POST	/api/auth/register	Public	Creates a new student account (rollNo, email, username, password).
POST	/api/auth/login	Public	Validates credentials, returns signed JWT & user profile.
GET	/api/problems/today	Student / Admin	Fetches the published Problem of the Day (POTD) for today's date.
GET	/api/problems	Student / Admin	Lists problems. Students see published only; Admins see all.
GET	/api/problems/:id	Student / Admin	Returns problem details, constraints, and sample inputs.
POST	/api/problems	Admin Only	Creates a new problem with description and test cases.
PUT	/api/problems/:id	Admin Only	Updates problem parameters, test cases, and status.
DELETE	/api/problems/:id	Admin Only	Removes problem from the catalog.

Method	Endpoint	Access Level	Description
GET	<code>/api/admin/dashboard</code>	Admin Only	Returns aggregated metrics: total students, problems, submissions.
GET	<code>/api/admin/students</code>	Admin Only	Directory of registered students with individual streaks and solves.
POST	<code>/api/submissions</code>	Student / Admin	Submits C solution code for a problem.
GET	<code>/api/submissions/my</code>	Student / Admin	Retrieves the current user's submission history.
GET	<code>/api/progress/my</code>	Student / Admin	Retrieves current user's streak, total solved, and total attempted.

6. Client Routing & Role-Based Access Control (RBAC)

The frontend enforces strict separation of concerns through layout nesting and route protection guards:

- **ProtectedRoute:** Inspects token existence in `AuthContext`. If the user possesses the `admin` role, they are automatically redirected to `/admin` to prevent accidental entry into student-specific workflows.
- **AdminRoute:** Validates that the active session belongs to an account with `user.role === 'admin'`. Regular students attempting to access `/admin/*` are redirected to `/dashboard`.
- **FallbackRoute:** Handles wildcard (*) routing cleanly. Unauthenticated users go to `/login`, students go to `/dashboard`, and administrators are directed to `/admin`.

7. Complete Project Directory Layout

```
Major_Project/ ├── backend/ | ├── configs/db.js # MongoDB connection setup | ──
```

controllers/ # Request handlers & logic | | └─ adminController.js # Dashboard & student

metrics | | └─ authController.js # Register, login | | └─ problemController.js # Problem

CRUD & POTD | | └─ progressController.js # User streaks & stats | | └─

submissionController.js # C code submissions | | └─ topicController.js # Topic

categorization | └─ middleware/ # authMiddleware, adminMiddleware | └─ models/ # User,

Problem, Submission, Progress, Topic | └─ routes/ # Express route declarations | └─

server.js # Application entry point | └─ frontend/ └─ src/ | └─ components/ | | └─

common/ # Button, Input, Loader | | └─ dashboard/ # StatCard, StreakCard | | └─ layout/ #

Navbar, Sidebar, StudentLayout, | | | # AdminNavbar, AdminLayout | | └─ problem/ #

CodeEditor, ProblemDescription | └─ context/AuthContext.jsx # Auth state & LocalStorage

sync | └─ pages/ | | └─ auth/ # Login, Register | | └─ student/ # Dashboard, Problems,

POTD, Submissions | | └─ admin/ # AdminDashboard, AdminProblems, | | # AdminStudents,

CreateProblem, EditProblem | └─ routes/AppRoutes.jsx # Complete application routing tree |

└─ services/ # Axios API abstraction services └─ vite.config.js # Vite & Tailwind

configuration

